Travaux Pratiques : Filtrage Multi-Critères, État Dérivé et Communication dans OmniMonitor

Objectifs

1. Centraliser l'état du parc de stations et des critères de filtrage dans le composant racine (`Lifting State Up`).
2. Calculer des indicateurs statistiques en temps réel (total, stations en ligne, stations en alerte) via l'état dérivé sans dupliquer `useState`.
3. Concevoir un composant `FilterBar` permettant de filtrer les stations par statut (`Toutes`, `En ligne`, `Hors ligne`) et par recherche textuelle instantanée.
4. Implémenter l'inversion de statut (`online` / `offline`) d'une station de manière immuable avec `.map()` et le *Spread Operator*.
5. Gérer proprement les rendus conditionnels (bannières d'anomalies sans le piège du `0`, et écran d'état vide avec réinitialisation des filtres).

Contexte du Projet : OmniMonitor - Centre de Contrôle Interactif

Les opérateurs de Météo-France disposent maintenant de la grille des stations issue du TP 2. Cependant, lorsque le parc compte de nombreuses stations, ils doivent pouvoir isoler immédiatement les stations hors ligne, rechercher une station par son nom, basculer une station en maintenance d'un simple clic et visualiser en haut d'écran les compteurs de santé du réseau qui se recalculent automatiquement.

Étape 1 : Enrichissement du Modèle de Données et État Racine (`App.jsx`)

1. Ouvrez `src/App.jsx` et enrichissez le tableau initial de votre `useState` avec la propriété `alerts` (nombre d'anomalies capteurs) et davantage de stations :

```
None
const INITIAL_STATIONS = [
  { id: 1, name: 'Toulouse-Blagnac', altitude: 152, status: 'online',
sensorsCount: 6, alerts: 0 },
  { id: 2, name: 'Pic du Midi', altitude: 2877, status: 'offline', sensorsCount:
3, alerts: 2 },
```

```
  { id: 3, name: 'Marseille-Marignane', altitude: 22, status: 'online',
sensorsCount: 5, alerts: 0 },
  { id: 4, name: 'Mont-Aigoual', altitude: 1567, status: 'online', sensorsCount:
8, alerts: 1 },
  { id: 5, name: 'Brest-Guipavas', altitude: 99, status: 'offline', sensorsCount:
4, alerts: 3 }
];
```

2. Dans `App.jsx`, ajoutez deux nouveaux états locaux pour piloter le filtrage :
    - `statusFilter` initialisé à `'all'` (valeurs possibles : `'all'`, `'online'`, `'offline'`).
    - `searchQuery` initialisé à `''` (chaîne vide).

Étape 2 : Implémentation de la Mutation Immuable de Statut (`toggleStatus`)

Jusqu'ici, vous pouviez ajouter et supprimer une station. Vous allez maintenant permettre de modifier un objet existant à l'intérieur du tableau d'état sans jamais le muter directement.

1. Dans `src/App.jsx`, créez une fonction `handleToggleStatus(id)` qui utilise `.map()` sur le tableau précédent pour inverser uniquement le statut de la station ciblée :

```
None
function handleToggleStatus(id) {
  setStations(prevStations =>
    prevStations.map(st =>
      st.id === id
        ? { ...st, status: st.status === 'online' ? 'offline' : 'online' }
        : st
    )
  );
}
```

2. Passez cette fonction `handleToggleStatus` en prop (`onToggleStatus`) à vos composants `<StationCard />`.
3. Dans `src/components/StationCard.jsx`, ajoutez un bouton permettant de déclencher `onToggleStatus(id)` dont le libellé s'adapte dynamiquement (`"Passer`

`Hors Ligne"` si la station est en ligne, ou `"Relancer la station"` si elle est hors ligne).

4. **Rendu conditionnel sécurisé :** Dans `StationCard.jsx`, affichez une alerte visuelle rouge (`"⚠️ " + alerts + " anomalie(s) active(s)"`) **uniquement** si `alerts > 0`. Vérifiez qu'aucun chiffre `0` parasite ne s'affiche sur la carte de `Toulouse-Blagnac` (`alerts: 0`).

Étape 3 : Calcul des États Dérivés (Statistiques et Filtrage)

Sans créer aucun `useState` supplémentaire, calculez directement dans le corps de `App.jsx` :

1. La liste filtrée `filteredStations` qui combine les deux critères (`statusFilter` ET `searchQuery` insensible à la casse) :

```
None
const filteredStations = stations.filter(st => {
  const matchesStatus = statusFilter === 'all' || st.status === statusFilter;
  const matchesSearch = st.name.toLowerCase().includes(searchQuery.toLowerCase());
  return matchesStatus && matchesSearch;
});
```

2. Les trois compteurs dérivés du parc global :
    - `totalStations = stations.length`
    - `onlineStations = stations.filter(st => st.status === 'online').length`
    - `alertStations = stations.filter(st => st.alerts > 0).length`
3. Transmettez ces trois compteurs en `props` à votre composant `src/components/Header.jsx` pour qu'il affiche en temps réel l'état de santé global du réseau en haut de la page.

Étape 4 : Création de la Barre de Filtres (`FilterBar.jsx`)

1. Créez un nouveau composant `src/components/FilterBar.jsx` recevant les props : `statusFilter`, `onStatusChange`, `searchQuery`, `onSearchChange` et `resultCount`.
2. Dans ce composant, affichez :

- Un champ de recherche `<input type="text" />` lié à `value={searchQuery}` et déclenchant `onSearchChange(e.target.value)` sur l'événement `onChange`.
- Trois boutons de filtre rapide : **Toutes** (`'all'`), **En ligne** (`'online'`) et **Hors ligne** (`'offline'`). Appliquez un style ou une classe distincte au bouton actuellement actif.
- Le nombre de stations affichées (`resultCount + " station(s) affichée(s)"`).

3. Importez et placez `<FilterBar />` dans `src/App.jsx` juste au-dessus de la grille des stations.

Étape 5 : Gestion de l'État Vide (*Empty State*)

1. Dans `src/App.jsx`, créez une fonction `handleResetFilters()` qui remet `statusFilter` à `'all'` et `searchQuery` à `''`.
2. Au niveau de l'affichage de la grille des stations dans `src/App.jsx` :
   - Si `filteredStations.length > 0`, affichez la grille des composants `<StationCard />` à partir de `filteredStations`.
   - Si `filteredStations.length === 0`, affichez un panneau d'état vide indiquant *"Aucune station ne correspond à votre recherche"* accompagné d'un bouton **"Réinitialiser les filtres"** qui appelle `handleResetFilters`.

Critères de Validation

Avant de passer au segment suivant (J1-S4 sur `useEffect`), vérifiez que :

- Aucun `useState` redondant n'a été créé pour stocker la liste filtrée ou les compteurs : tout est calculé en état dérivé.
- Le clic sur le bouton de changement de statut d'une carte met instantanément à jour son badge, les compteurs du `Header` et la liste filtrée si un filtre de statut est actif.
- Les stations ayant `alerts: 0` n'affichent pas le chiffre `0` dans leur carte.
- Taper un nom inexistant dans la barre de recherche affiche proprement l'écran d'état vide et le bouton de réinitialisation rétablit toute la liste.